Общество с ограниченной ответственностью
«Центр повышения квалификации и профессиональной подготовки «Основание»

ВЫПУСКНАЯ АТТЕСТАЦИОННАЯ (ПРОЕКТНАЯ) РАБОТА

Разработка программного обеспечения (компьютерной программы) для поддержания актуальной информации о курсах криптовалют по отношению к доллару США

по программе профессиональной переподготовки

«Азбука цифры. Python-программист»

Выполнил:  03.11.2026 А.П.Пидкасистый

подпись *дата* *инициалы, фамилия*

Москва 2026

Оглавление

1. ВВЕДЕНИЕ....3
2. ТЕХНИЧЕСКОЕ ЗАДАНИЕ....3
2.1 Общие сведения....3
2.1.1 Полное наименование системы и ее условное обозначение....3
2.1.2 Организация — разработчик и организация — заказчик....3
2.2 Назначение и цели создания системы....3
2.2.1 Перечень работ по созданию системы....3
2.2.2 Цели создания системы....4
2.2.3 объект информатизации....4
2.2.4 Сведения о существующей технологии....4
2.3 Требования к программному обеспечению pcryptoboard....4
2.3.1 Функциональные требования....4
2.4 Требования к документации....4
2.5 Порядок предоставления дистрибутива....5
2.6 Защита исходного кода....5
2.7 Требования к техническому обеспечению....5
2.8 Глоссарий и перечень сокращений....5
3. РЕЗУЛЬТАТЫ ИССЛЕДОВАНИЯ ОТКРЫТЫХ API, ПРЕДОСТАВЛЯЮЩИХ КУРСЫ КРИПТОВАЛЮТ....5
3.1 Поиск открытых API....5
3.2 Поиск API с курсами криптовалют....6
3.2 Выбор конкретного API для использования в проекте....6
4 ТРЕБОВАНИЯ К ГРАФИЧЕСКОМУ ИНТЕРФЕЙСУ ПОЛЬЗОВАТЕЛЯ....7
4.1 Требования к интерфейсу....7
4.2 Расположение элементов интерфейса....11
4.3 Расположение файлов программы....11
5. ТРЕБОВАНИЯ К ТЕСТИРОВАНИЮ....11
6 ЛИСТИНГ....12
7. ЗАКЛЮЧЕНИЕ....32

1. ВВЕДЕНИЕ

Программа для поддержания актуальной информации о курсах криптовалют по отношению к доллару США (далее по тексту “программа «Курсы криптовалют»” или “Приложение” или “ПО”) разрабатывается для пользователей, которым ежедневно необходима актуальная информация о курсах необходимых им криптовалют. Приложение позволяет в любой момент времени получать актуальные курсы избранных пользователем криптовалют с использованием сервиса https://www.coinlore.com. Приложение работает на персональных компьютерах под управлением операционной системы Windows 10. Для работы приложения на персональном компьютере должен быть установлен компилятор языка Python версии не ниже 3.13. Приложение имеет интерфейс на русском языке и не предназначено для распространения за пределами Российской Федерации. К работе с приложением допускаются пользователи не младше 14 лет.

2. ТЕХНИЧЕСКОЕ ЗАДАНИЕ

2.1 Общие сведения

2.1.1 Полное наименование системы и ее условное обозначение

Программное обеспечение: «Курсы криптовалют».

Условное обозначение разрабатываемой информационной системы — **pcryptoboard**.

Логотип: 

2.1.2 Организация — разработчик и организация — заказчик

Разработчик: налогоплательщик налога на профессиональный доход Пидкасистый Александр Павлович.

Заказчик: ООО “Основание”.

2.2 Назначение и цели создания системы

2.2.1 Перечень работ по созданию системы

Данным техническим заданием требуется создать ПО pcryptoboard и произвести следующие работы:

- Разработать функционал отображения пользовательского интерфейса;
- Разработать функционал обращения к сайту https://www.coinlore.com;

- Разработать функционал вывода полученных данных о курсах криптовалют в интерфейс пользователя;
- Обеспечить обработку исключений для повышения стабильности работы приложения.

2.2.2 Цели создания системы

Программное обеспечение программа “Курсы криптовалют” (далее — pcryptoboard) разрабатывается в целях обеспечения пользователей актуальной информацией о курсах выбранных ими криптовалют.

2.2.3 объект информатизации

Объектом автоматизации является работа программного обеспечения “Курсы криптовалют”.

2.2.4 Сведения о существующей технологии

В настоящее время для получения информации о курсах криптовалют пользователям необходимо открыть браузер, найти сайт, отображающий курсы интересуемых пользователя криптовалют, в длинном списке криптовалют найти нужные и периодически повторять это действие для получения актуальной информации. Программное обеспечение pcryptoboard призвано помочь пользователям иметь актуальные курсы единожды выбранных криптовалют на экране своего компьютера без лишних усилий.

2.3 Требования к программному обеспечению pcryptoboard

2.3.1 Функциональные требования

ПО pcryptoboard должно предоставлять пользователю следующие возможности:

- при запуске ПО в главном окне программы отображать курсы избранных криптовалют;
- формировать список избранных криптовалют путем выделения нужных в полном списке криптовалют, отсортированных по алфавиту;
- добавлять/ заменять криптовалюты в избранном списке;
- обновлять текущие курсы криптовалют вручную и автоматически с установленным интервалом;
- периодически обновлять справочник всех доступных криптовалют вручную для включения в него новых криптовалют и удаления ушедших с рынка;
- сохранять на диске и использовать по мере необходимости справочник криптовалют, список избранных криптовалют и другие настройки программы;
- обеспечить взаимодействие с пользователем на русском языке.

2.4 Требования к документации

В момент сдачи проекта заказчику предоставляется следующий набор документов:

- Руководство пользователя в электронном виде;
- Справочник с названиями популярных криптовалют.

2.5 Порядок предоставления дистрибутива

По окончании разработки Исполнитель должен предоставить Заказчику дистрибутив системы в составе:

- архив с исходными кодами всех программных модулей и разделов, файлами данных;
- файл проекта в формате *.py
- дистрибутив Python версии 3.13
- пароль от архива

2.6 Защита исходного кода

Требуется защитить исходный код. Архив с исходными кодами должен быть заархивирован с паролем длиной не менее 10 символов.

2.7 Требования к техническому обеспечению

Требования к ПК:

- Компьютер с процессором Процессор Intel® Core™ i3 2,3 ГГц
- ОЗУ 4096 Мб (рекомендуется от 8192 Мб)
- Место на жестком диске от 1 Гб.

2.8 Глоссарий и перечень сокращений

ПО — программное обеспечение

ПК — персональный компьютер

ОЗУ — оперативное запоминающее устройство (оперативная память ПК)

3. РЕЗУЛЬТАТЫ ИССЛЕДОВАНИЯ ОТКРЫТЫХ API, ПРЕДОСТАВЛЯЮЩИХ КУРСЫ КРИПТОВАЛЮТ

3.1 Поиск открытых API

Для поиска открытых API использовалась поисковые системы Google и Яндекс. По запросу “public api list” было найдено более 10 сайтов с открытыми API. Ниже приведены ссылки на первые 5 сайтов со списками открытых API:

https://github.com/public-apis/public-apis

https://rapidapi.com/collection/list-of-free-apis

https://apislist.com/

https://publicapis.dev/

https://apilist.fun/

3.2 Поиск API с курсами криптовалют

При анализе найденных сайтов и поиске новых по запросу «api курсы криптовалют» были найдены следующие API, позволяющие получить курсы обмена криптовалюты и не требующие ключа apiKey:

Доступ через виртуальные частные сети (Virtual Private Network, VPN), подменяющие российское местонахождение источника запроса:

https://publicapis.dev/category/cryptocurrency

https://coinmarketcap.com/api/

Прямой доступ (без VPN):

https://docs.coingecko.com/docs/keyless-public-api

https://www.coinlore.com/ru/cryptocurrency-data-api

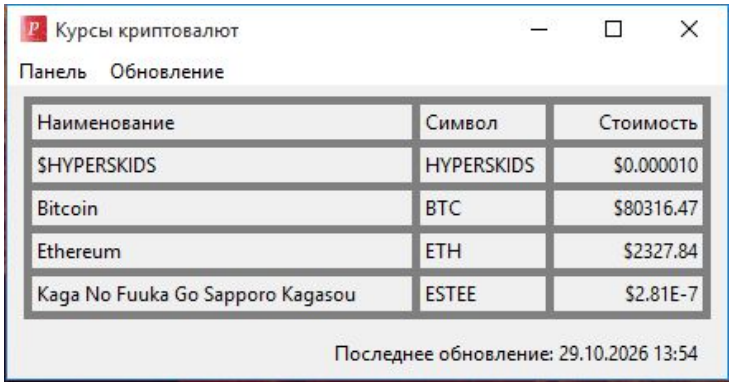
3.2 Выбор конкретного API для использования в проекте

Был проведен анализ отзывов в интернет по всем исследуемым сервисам, в результате для использования в проекте был выбран сервис https://www.coinlore.com с большим количеством наименований криптовалют (около 15 тыс. наименований), наиболее стабильный и доступный в любое время суток, и не закрытый для ip-адресов из России.

4 ТРЕБОВАНИЯ К ГРАФИЧЕСКОМУ ИНТЕРФЕЙСУ ПОЛЬЗОВАТЕЛЯ

4.1 Требования к интерфейсу

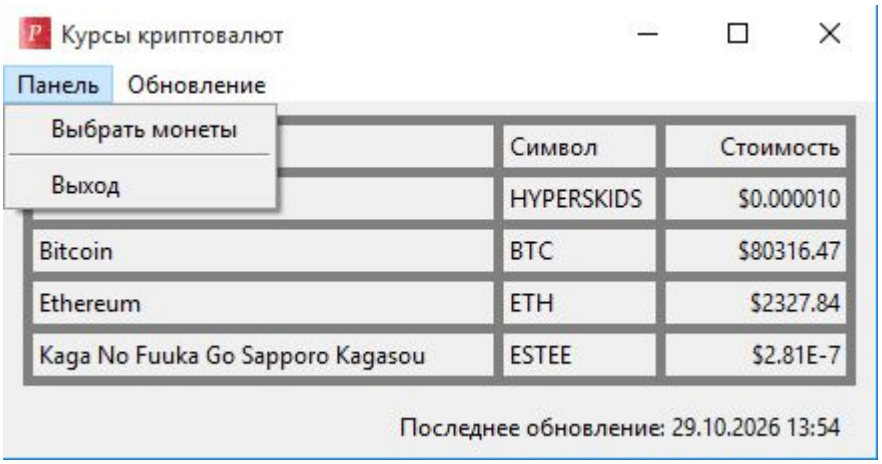
Интерфейс ПО pcryptoboard должен соответствовать приведенному эскизу:



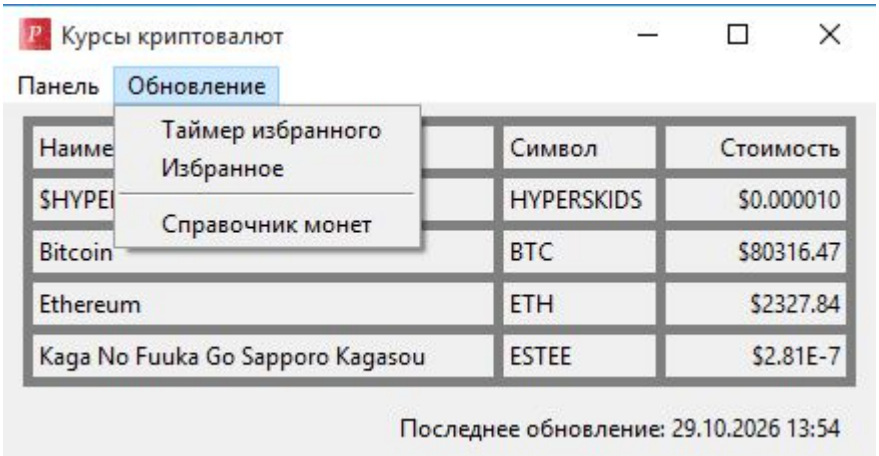
При двойном клике по свободной части окна должна скрываться строка меню:



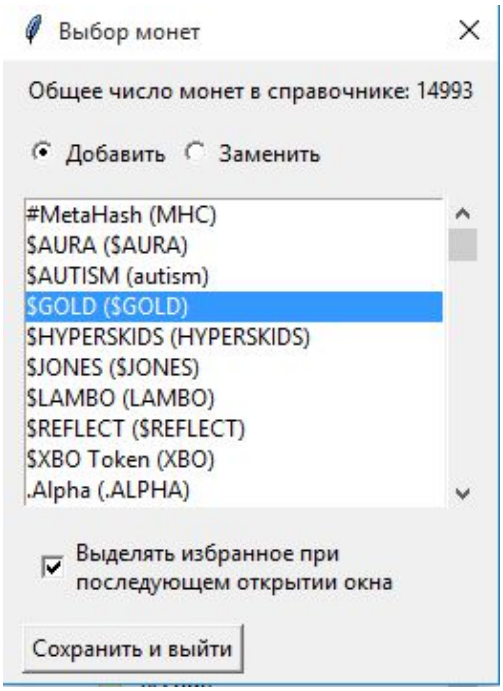
При нажатии мышкой пункта меню «Панель», должен раскрываться список с пунктами меню «Выбрать монеты» для формирования списка избранных криптовалют и «Выход» для закрытия Приложения.



При нажатии мышкой пункта меню «Обновление», должен раскрываться список с пунктами меню «Таймер избранного», «Избранное» и «Справочник монет».



Выбор пункта меню «Панель»/ «Выбрать монеты» должен открывать окно для формирования списка выбранных криптовалют.



В открытом окне должно быть представлено:

- общее число криптовалют, из которых можно формировать Избранное;
- переключатель «Добавить/Заменить», позволяющий при сохранении и выходе из режима добавить или заменить ранее выбранные криптовалюты на отмеченные в списке;
- окно со списком криптовалют, в котором можно выделить несколько позиций для добавления или замены ранее избранных;
- переключатель, установка/снятие которого позволяет при следующем входе в режим получить выделение избранных криптовалют или списка с неотмеченными позициями;
- кнопка, при нажатии на которую сохраняется выбор пользователя и осуществляется закрытие окна;

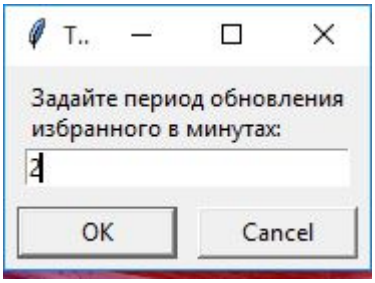
- крестик в правом верхнем углу, нажатие на который приводит к закрытию окна без сохранения выбора пользователя.

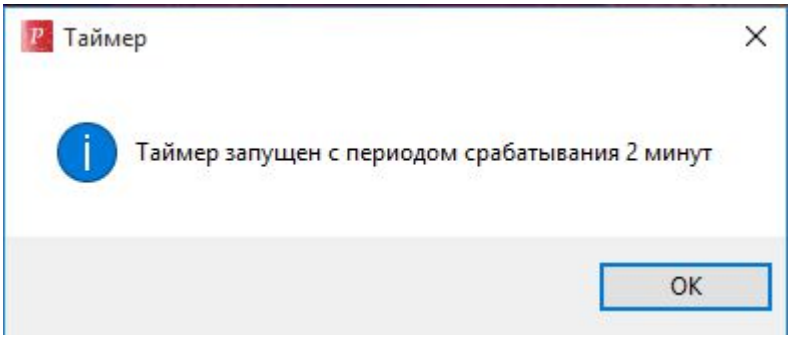
Выбор пользователя сохранять / или не сохранять выделение избранных криптовалют в общем списке валют при следующем входе в режим (положение переключателя) должен сохраняться вне зависимости от способа выхода из режима (кнопка или крестик в правом верхнем углу окна).

Выбор пункта меню «Обновление» / «Избранное» должен запускать процесс обновления курсов избранных криптовалют. Итогом обновления избранного должно быть:

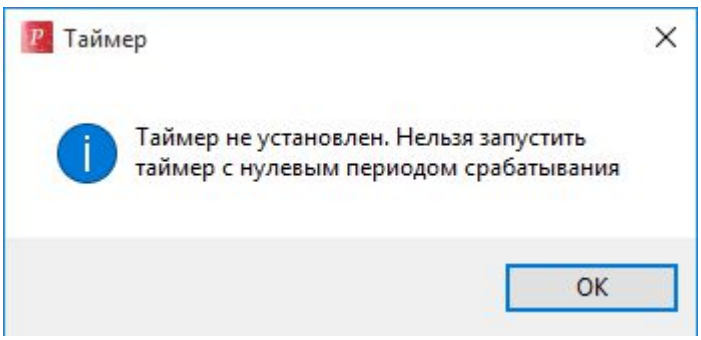
- файл data/indices_coins.json с актуальным перечнем криптовалют,
- отображение обновленных курсов на панели,
- отображение даты и времени последнего обновления в правом нижнем углу главного окна.

Выбор пункта меню «Обновление» / «Таймер избранного» должен позволить задать период обновления Избранного и запустить в фоновом режиме таймер, который с заданной периодичностью будет запускать обновление Избранного в автоматическом режиме:

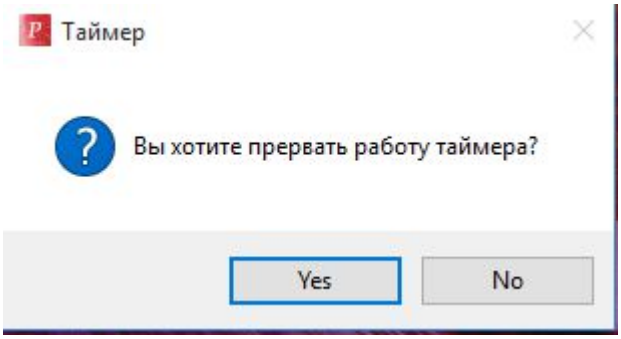




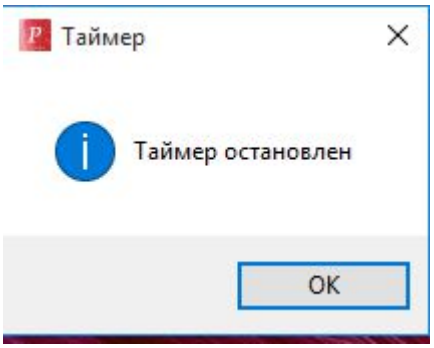
Период обновления задается в диапазоне от 0 до 240 минут. При вводе нуля, программа должна предупредить пользователя о невозможности запустить таймер и выйти из режима



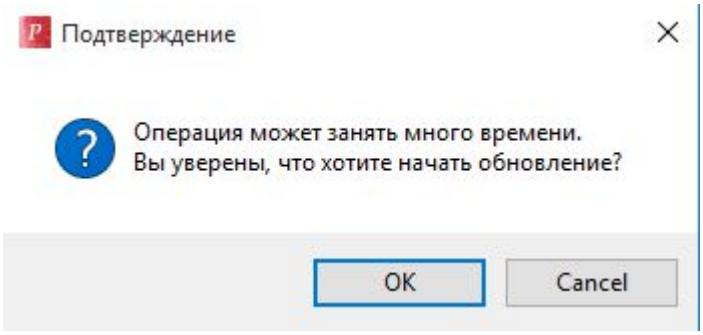
В случае, когда таймер уже запущен и работает автоматическое обновление избранных криптовалют, выбор пункта меню «Обновление» / «Таймер избранного» должен приводить к завершению автоматического обновления.



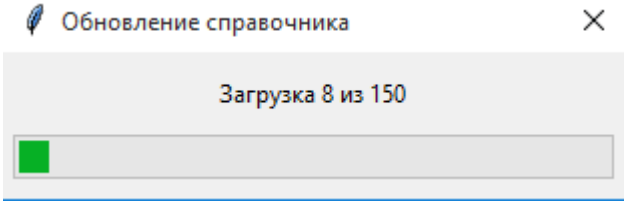
И информированию пользователя о прекращении этого процесса:



Выбор пункта меню «Обновление» / «Справочник монет» должен предупредить пользователя о длительности процесса обновления Справочника криптовалют и, только после подтверждения пользователем этой операции, запустить в фоновом режиме процесс обновления Справочника и сделать неактивным до следующего запуска Программы этот пункт меню.



Процесс обновления Справочника должен сопровождаться индикатором прогресса, как показано на рисунке ниже:



На сервере https://www.coinlore.com отслеживается около 15 тыс. наименований криптовалют. По правилам, установленным на сервере, за один запрос можно получать до 100

наименований включительно. Последующий запрос должен осуществляться не ранее, чем через одну секунду после предыдущего. Таким образом, шкала индикатора прогресса должна быть привязана к максимальному числу планируемых запросов, и текст, выводимый на окне индикатора прогресса, должен показывать номер обрабатываемого запроса из общего числа запросов к серверу.

Процесс обновления Справочника должен происходить в фоновом режиме и не мешать пользователю пользоваться в это же время другими возможностями программы. Итогом обновления Справочника должен быть файл data/total_coins.json с актуальным перечнем криптовалют.

4.2 Расположение элементов интерфейса

Расположение элементов интерфейса представлено на эскизах выше. На главном окне расположение следующее:

- заголовок окна — “Курсы криптовалют”;
- строка меню из двух каскадных меню: «Панель» и «Обновление»
- раскрывающееся меню «Панель»: «Выбрать монеты» и «Выход»;
- раскрывающееся меню «Обновление»: «Таймер избранного», «Избранное» и «Справочник монет»;

На всплывающем окне для выбора криптовалют расположение следующее:

- заголовок окна – «Выбор монет»;
- текстовая метка с общим количеством криптовалют;
- переключатель «Добавить/Заменить»;
- окно со списком всех криптовалют в алфавитном порядке и прокруткой;
- переключатель для выделения/ снятия выделения с избранного при последующем открытии окна;
- кнопка с надписью — “Сохранить и выйти”

На окне с индикатором прогресса:

- текстовая метка с номером текущего запроса и общим числом запросов;
- индикатор прогресса.

4.3 Расположение файлов программы

Все файлы данных, сохраняемых программой, должны располагаться в папке /data, расположенной в том же месте, что и сама программа. Файл с логотипом программы располагается в корневой папке проекта.

5. ТРЕБОВАНИЯ К ТЕСТИРОВАНИЮ

5.1 ПО pcryptoboard должно быть протестировано перед отправкой Заказчику. Тестирование проводится в соответствии с “Программой и методикой испытаний”.

В отсутствие необходимых файлов с данными (или самой папки /data) Программа должна предупреждать пользователя о таком отсутствии, продолжать устойчиво работать и создавать нужные ей файлы с данными.

6 ЛИСТИНГ

```
from tkinter import *
from tkinter import messagebox as mb
from tkinter import ttk
from tkinter import simpledialog
import json
import os
import requests
from datetime import datetime
import pickle
import time
import queue
import threading


def quit(event = None):
    if thrd:                # Если запущен поток, то
        stop_event.set()    # устанавливаем флаг останова потока
        thrd.join()         # ожидаем окончания работы потока
    root.destroy()          # Закрываем главное окно и завершаем работу программы


def hide_menu(event):
    # Скрываем рамку, строку заголовка и меню окна
    global menu_flag
    empty_menu = Menu(root)         # создаём пустое меню
    root.config(menu=empty_menu)    # заменяем действующее меню на пустое
```

```
    root.overrideredirect(True)     # удаляем рамку и строку заголовка
    menu_flag = False               # меняем флаг корневого окна


def show_menu(event):
    # Восстанавливаем рамку, строку заголовка и меню окна
    global menu_flag
    root.config(menu=menu_bar)      # возвращаем действующее меню
    root.overrideredirect(False)    # возвращаем рамку и строку заголовка
    menu_flag = True                # меняем флаг корневого окна



def clear_frame_root():
    # Функция очистки главного окна от виджетов
    if frame_root.winfo_children():             # Проверяем наличие виджетов в главном окне
        for widget in frame_root.winfo_children():  # Если виджеты есть,
            widget.destroy()                    # то удаляем их
    if frame_footer.winfo_children():               # Проверяем наличие виджетов в футоре
        for widget in frame_footer.winfo_children():    # Если виджеты есть,
            widget.destroy()                        # то удаляем их


def on_dashboard():
    # Функция создания виджетов в главном окне: выводятся избранные криптовалюты и их курсы
    global FAVOURITES_COINS_FNAME
    global time_stamp
    clear_frame_root()  # Очищаем главное окно от виджетов
    favourites = load_json_safely(FAVOURITES_COINS_FNAME) # Загружаем избранное
    if favourites == None:
        return
    try:
```

```
            with open(TIME_STAMP_FNAME, 'r', encoding='utf-8') as f:  # Загружаем штамп времени
                time_stamp = f.read()
        except Exception as e:
            mb.showerror("Ошибка", f"При попытке открыть файл {TIME_STAMP_FNAME} произошла ошибка {e}")

        # Подбор ширины столбцов по содержимому
        m_name = len(max(favourites, key=lambda coin: len(coin["name"]))["name"])
        name_width = (m_name if m_name > 15 else 15)
        m_symbol = len(max(favourites, key=lambda coin: len(coin["symbol"]))["symbol"])
        symb_width = (m_symbol if m_symbol > 8 else 8)
        m_price = len(max(favourites, key=lambda coin: len(coin["price_usd"]))["price_usd"])
        pric_width = (m_price if m_price > 12 else 12)

        # Вывод информации на дашборд
        Label(frame_root, text="Наименование", compound=LEFT, width=name_width,
           justify="left", anchor="w").grid(row=0, column=0, padx=(5,0), pady=5, sticky ="w")
        Label(frame_root, text="Символ", compound=LEFT, width=symb_width,
           justify="left", anchor="w").grid(row=0, column=1, padx=(5,0), pady=5, sticky ="w")
        Label(frame_root, text="Стоимость", compound=LEFT, width=pric_width,
           justify="left", anchor="e").grid(row=0, column=2, padx=5, pady=5, sticky ="w")
            for i, coin in enumerate(favourites): # Выводим информацию об избранных монетах
            Label(frame_root, text=f"{coin["name"]}", compound=LEFT, width=name_width,
                justify="left", anchor="w").grid(row=i+1, column=0, padx=(5,0), pady=(0,5), sticky ="w")
            Label(frame_root, text=f"{coin["symbol"]}", compound=LEFT, width=symb_width,
                justify="left", anchor="w").grid(row=i+1, column=1, padx=(5,0), pady=(0,5), sticky ="w")
            Label(frame_root, text=f"${coin["price_usd"]}", compound=LEFT, width=pric_width,
              justify="left", anchor="e").grid(row=i+1, column=2, padx=5, pady=(0,5), sticky ="w")
```

```
        label_upd = Label(frame_footer, text=f"Последнее обновление: {time_stamp}")
        label_upd.pack(side=RIGHT, pady=5, padx=5)
    # on_dashboard


    def save_pickle_safely(data, filename):
        # Функция преобразует объект в байтовый поток и сериализует поток в файл с именем filename
        global locker
        try:
            # Создаём директорию, если её нет
            os.makedirs(os.path.dirname(os.path.abspath(filename)), exist_ok=True)
            with locker: # Устанавливаем блокировку перед критической секцией (во избежания конфликтов за ресурсы)
                # Сохраняем файл
                with open(filename, 'wb') as f:
                    pickle.dump(data, f)
            return True
        except (PermissionError, OSError) as e:
            mb.showerror("Ошибка", f"Ошибка записи файла: {str(e)}")
            return False
        except Exception as e:
            mb.showerror("Ошибка", f"Ошибка: {str(e)}")
            return False


    def save_json_safely(data, filename):
        # Функция сериализует данные из data в файл с именем filename
        global locker
        try:
            # Создаём директорию, если её нет
            os.makedirs(os.path.dirname(os.path.abspath(filename)), exist_ok=True)
            with locker: # Устанавливаем блокировку перед критической секцией (во избежания конфликтов за ресурсы)
```

```
            # Сохраняем файл
            with open(filename, 'w', encoding='utf-8') as f:
                json.dump(data, f, indent=4, ensure_ascii=False)
            return True
        except TypeError as e:
            mb.showerror("Ошибка", f"Ошибка типа данных: {str(e)}")
            return False
        except (PermissionError, OSError) as e:
            mb.showerror("Ошибка", f"Ошибка записи файла: {str(e)}")
            return False


    def load_json_safely(filename):
        # Функция возвращает данные из файла с именем filename
        global locker
        try:
            with locker: # Устанавливаем блокировку перед критической секцией (во избежания конфликтов за ресурсы)
                with open(filename, 'r', encoding='utf-8') as f:
                    return json.load(f)
        except FileNotFoundError:
            mb.showerror("Ошибка", f"Файл {filename} не найден")  # Возвращает список (None "ok")
            return None
        except json.JSONDecodeError:
            mb.showerror("Ошибка", "Ошибка декодирования JSON")  # Возвращает список (None "ok")
            return None


    def load_pickle_safely(filename):
        # Функция возвращает байтовые данные из файла с именем filename
        try:
```

```
            with locker: # Устанавливаем блокировку перед критической секцией (во избежания конфликтов за ресурсы)
                with open(filename, 'rb') as f:
                    return pickle.load(f)
            except FileNotFoundError:
                mb.showerror("Информация", f"Файл {filename} не найден")  # Возвращает список (None "ok")
                return None
            except Exception as e:
                mb.showerror("Ошибка", f"Ошибка: {str(e)}")
                return None


        def on_favourites():
            selected_indices = set()  # Множество для хранения списка выбранных индексом
            sel_buffer = []          # Буфер для хранения списка выделенных элементов виджета lb
            sel_ind_flag = False      # Флаг отображения выбранных индексов при открытии окна modal_window


            def chk_btn_changed():
                nonlocal sel_ind_flag
                sel_ind_flag = chk_btn_var.get()


            def on_select(event):
                # Функция сохраняет список выделенных элементов виджета lb в буфере
                nonlocal sel_buffer
                sel_buffer = event.widget.curselection()


            def on_click():
                # Функция устанавливает новые значения для множества selected_indices
                # и завершает работу окна
                nonlocal selected_indices  # Оъявляем переменную не локальной, доступной для изменения
```

```
    nonlocal sel_buffer
    if how_to_set.get():
        selected_indices.update(sel_buffer)     # добавляем
    else:
        selected_indices.clear
        selected_indices = set(sel_buffer)      # заменяем
    modal_window.destroy()

# Читаем из файла большой перечень валют и сохраняем в переменную tot_coins
tot_coins = load_json_safely(TOTAL_COINS_FNAME)
if tot_coins == None:
    return   # Если файл не найден, то выход из функции

# Читаем из файла индексы избранных валют и сохраняем в переменную selected_indices
sel_buffer = load_pickle_safely(SELECTED_INDS)
if sel_buffer != None:
    selected_indices = set(sel_buffer)
sel_buffer = []

# Читаем из файла флаг отображения выбранных индексов при открытии окна
sel_ind_flag = load_pickle_safely(FAVOURITES_VIEW)
if sel_ind_flag == None:
    sel_ind_flag = False

# Создаём дочернее модальное окно с настройками.
modal_window = Toplevel(root)
modal_window.title("Выбор монет")

# Создаем фреймы
frame_lab = Frame(modal_window)
```

```
frame_lab.pack(fill="x", padx=10, pady=5)

frame_rbtn = Frame(modal_window)

frame_rbtn.pack(fill="x", padx=10, pady=5)

frame_modal_w = Frame(modal_window)

frame_modal_w.pack(fill="x", padx=10, pady=5)

frame_chk = Frame(modal_window)

frame_chk.pack(fill="x", padx=10, pady=5)

frame_buttons = Frame(modal_window)

frame_buttons.pack(fill="x", padx=10, pady=5)


# Создаем метку с общим числом монет

        Label(frame_lab, text=f"Общее число монет в справочнике: {len(tot_coins)}").pack(side=LEFT)


# Создаем переключатель "Заменить/Добавить"

how_to_set = BooleanVar(value=True)

        Radiobutton(frame_rbtn, text='Добавить', variable=how_to_set, value=True).pack(side=LEFT)

        Radiobutton(frame_rbtn, text='Заменить', variable=how_to_set, value=False).pack(side=LEFT)


# Создаём вертикальную Scrollbar

scrollbar = Scrollbar(frame_modal_w, orient=VERTICAL)

scrollbar.pack(side=RIGHT, fill=Y)


# Создаем Listbox с названием криптовалюты для их выбора

lb = Listbox(frame_modal_w, selectmode="multiple", yscrollcommand=scrollbar.set)

lb.pack(fill="both", expand=True)


# Настраиваем Scrollbar, чтобы он управлял прокруткой Listbox

scrollbar.config(command=lb.yview)
```

```
        # Заполняем Listbox элементами
        indices = []                                # Список для хранения идентификаторов избранных валют
        for i, coin in enumerate(tot_coins):
            lb.insert(END, f"{coin["name"]} ({coin["symbol"]})")
            if sel_ind_flag and i in selected_indices:  # Если i совпадает с одним из элементов множества selected_indices, то
                lb.select_set(i)                    # Этот элемент выделяется

        # Обработка события изменения выбора элементов lb
        lb.bind("<<ListboxSelect>>", on_select)

        chk_btn_var = BooleanVar(value=sel_ind_flag)
        chk_btn = Checkbutton(frame_chk, text="Выделять избранное при\nпоследующем открытии окна", justify="left", anchor="w", variable=chk_btn_var,
                              onvalue=True, offvalue=False, command=chk_btn_changed)
        chk_btn.pack(side=LEFT, padx=5)

        btn_save = Button(frame_buttons, text="Сохранить и выйти", command=on_click)
        btn_save.pack(side=LEFT)

        # Настраиваем модальность
        modal_window.transient(root)    # Связываем с главным окном
        modal_window.grab_set()         # Захватываем ввод
        modal_window.focus_set()        # Переводим фокус на окно
        modal_window.wait_window()      # Ждём закрытия окна

        # Сохраняем множество со списком выбранных индексов
        save_pickle_safely(selected_indices, SELECTED_INDS)

        # Сохраняем выбор "Выделять избранные монеты при последующем открытии окна"
        save_pickle_safely(sel_ind_flag, FAVOURITES_VIEW)
```

```
        # Формируем список идентификаторов избранных валют
        for i, coin in enumerate(tot_coins):
            if i in selected_indices:            # Если i совпадает с одним из элементов множества selected_indices, то
                indices.append(coin["id"])      # добавляем i-й идентификатор валюты в избранное
        indices_str = ','.join(indices)         # Формируем строку для подстановки в запрос к сайту

        # Сохраняем строку для запроса информации по избранным валютам в файл
        try:
            with open(FAVOURITES_ID_FNAME, 'w', encoding='utf-8') as f:
                f.write(indices_str)
        except Exception as e:
                        mb.showerror("Ошибка", f"При попытке сохранить файл {FAVOURITES_ID_FNAME} произошла ошибка {e}.")

        # Обновляем избранные валюты и перерисовывам главное окно
        update_favor()

    # def on_favourites


    def update_favor():
        global fav_id
        global time_stamp
        global FAVOURITES_ID_FNAME
        # Обновление избранных криптовалют
        try:
            with open(FAVOURITES_ID_FNAME, 'r', encoding='utf-8') as f:
                fav_id = f.read()
        except Exception as e:
```

```
            mb.showerror("Ошибка", f"При попытке открыть файл {FAVOURITES_ID_FNAME} произошла ошибка {e}.\nВ меню \'Настройки/Справочник\' выберите отслеживаемые валюты")
        if fav_id != '':
            data = response(FAVOURITES_COINS, params={"id": fav_id}, timeout=(10, 30)) # Делаем запрос к сайту по выбранным валютам
            if data == None:    # Если запрос неудачный, то
                return          # функция завершает работу
            save_json_safely(data.json(), FAVOURITES_COINS_FNAME)
            time_stamp = datetime.now().strftime("%d.%m.%Y %H:%M")
            try:
                with open(TIME_STAMP_FNAME, 'w', encoding='utf-8') as f:
                    f.write(time_stamp)
            except Exception as e:
                mb.showerror("Ошибка", f"При попытке открыть файл {TIME_STAMP_FNAME} произошла ошибка {e}")
            on_dashboard()  # Перерисовываем виджеты главного окна. В будущем сделать обновление, а не перерисовку
        else:
            mb.showerror("Ошибка", "Не выбрано ни одной валюты")
    # update_favor

    def update_guide_dialog():
        # Функция обновления Справочника криптовалют
        global thrd

        def progress_window_quit(event=None):
            global stop_event, thrd
            nonlocal progress_window
            if thrd:                    # Если запущен поток, то
                stop_event.set()        # устанавливаем флаг останова потока в True,
                thrd.join()             # ожидаем окончания работы потока,
                stop_event.clear()      # устанавливаем флаг останова потока в False,
```

```
            thrd = None             # и устанавливаем указатель в None
        progress_window.destroy()   # Закрываем главное окно и завершаем работу программы


    # Диалог для запуска процедуры обновления справочника валют
        rsansw = mb.askokcancel("Подтверждение", "Операция может занять много времени.\nВы уверены, что хотите начать обновление?")
    if not rsansw:
        return          # Если запрос неудачен, то выход
     update_menu.entryconfig("Справочник монет", state="disabled") # Делаем пункт меню неактивным до следующего запуска программы
    # Создаем окно для отображения Индикатора прогресса
    progress_window = Toplevel(root)
    progress_window.title("Обновление справочника")


    # Создаём очередь для обмена данными
    q = queue.Queue()


    # Создание сайзера
    frame_progress = Frame(progress_window)
    frame_progress.pack(fill="x", padx=5, pady=5)


    # Создание метки для вывода цифровых значений индикатора прогресса
    label = Label(frame_progress, text="Загрузка...")
    label.pack(pady=5)


    # Создание индикатора загрузки
    progress_bar = ttk.Progressbar(
        frame_progress,
        orient='horizontal',
        length=300,
        mode='determinate'  # Определённый режим (точный прогресс)
```

```
        )
        progress_bar.pack(pady=5)

        # Связываем обработчик события
        progress_window.bind('<<Updated>>', lambda e, qu=q, pb=progress_bar, lb=label: on_update(e, qu, pb, lb))
        progress_window.bind('<<Event_close_window>>', progress_window_quit)

        # Запускаем поток
        thrd = threading.Thread(target=worker_in_thread, args=(progress_window, q, progress_bar, label))

        # Запускаем функцию через 100 миллисекунд после создания окна
        progress_window.after(100, lambda : thrd.start())

        # Настраиваем окно
        progress_window.transient(root)    # Связываем с главным окном
        progress_window.protocol("WM_DELETE_WINDOW", progress_window_quit)
    # update_guide_dialog

    def worker_in_thread(win, q, pb, lb):
        # Функция для выполнения в потоке. Параметры: win - ссылка на окно с индикатором прогресса, pb - ссылка на виджет
        # индикатора прогресса, lb - ссылка на метку для вывода цифровых значений индикатора прогресса, q - ссылка на
        # очередь для обмена данными.
        if stop_event.is_set():  # Если установлен флаг останова потока, то завершаем функцию
            return
        r = response(REQUEST_TOTAL_COINS, params={"start":0, "limit":100}, timeout=(10, 30)) # Делаем запрос к сайту по валютам
        if r == None:
            return
```

```
        ass_data = r.json()["data"]
        ass_info = r.json()["info"]["coins_num"]  # Для постоянной работы
        # ass_info = 450  # Для отладки

        count = ass_info // 100
        rest = ass_info % 100
        max_count = count if rest == 0 else count + 1  # Вычисляем maximum для ProgressBar
        q.put((1, max_count))
        # Генерируем событие для обновления виджета
        win.event_generate('<<Updated>>', when='tail')
        time.sleep(2)  # Задержка 2 сек., чтобы не перегужать сервер

        for i in range(1, count):
            if stop_event.is_set():  # Если установлен флаг останова потока, то завершаем функцию
                return
            r = response(REQUEST_TOTAL_COINS, params={"start":i * 100, "limit":100}, timeout=(10, 30)) # Делаем запрос к сайту по валютам
            tmp_ass = r.json()["data"]
            ass_data = ass_data + tmp_ass
            q.put((i + 1, max_count))
            win.event_generate('<<Updated>>', when='tail')
            time.sleep(2)  # Задержка 2 сек., чтобы не перегужать сервер

        if stop_event.is_set():  # Если установлен флаг останова потока, то завершаем функцию
            return
        if rest != 0:
            r = response(REQUEST_TOTAL_COINS, params={"start":count * 100, "limit":rest}, timeout=(10, 30))
            tmp_ass = r.json()["data"]
            ass_data = ass_data + tmp_ass
            q.put((max_count, max_count))
```

```
            win.event_generate('<<Updated>>', when='tail')


        # Сортировка по имени
        sorted_ass = sorted(ass_data, key=lambda x: x["name"])


        # Сохраняем в файл, чтобы не делать запросы слишком часто
        save_json_safely(sorted_ass, TOTAL_COINS_FNAME)


        # Генерируем событие для закрытия окна с индикатором прогресса
        win.event_generate('<<Event_close_window>>', when='tail')
    # worker_in_thread


    def on_update(event, q, pb, lb):
        # Получаем значение из очереди и обновляем прогресс-бар
        tmp = q.get()
        pb['value'] = tmp[0]
        pb['maximum'] = tmp[1]
        lb.config(text=f"Загрузка {tmp[0]} из {tmp[1]}")
    # on_update


    def response(*args, **kwargs):
        # Формирует запрос к серверу и возвращает данные в формате JSON. Параметры: url - адрес сервера,
        # prm - параметры запроса, tm - кортеж с максимальным временем ожидания соединения с сервером и
        # максимальным временем ожидания получения данных
        try:
            r = requests.get(*args, **kwargs)   # Формируем запрос и получаем данные
            r.raise_for_status()                # выбросит HTTPError для кодов 4xx/5xx
            return r
        except requests.Timeout:
```

```
            mb.showerror("Ошибка", "Запрос превысил время ожидания")
            return None
        except requests.ConnectionError:
            mb.showerror("Ошибка", "Не удалось соединиться с сервером")
            return None
        except requests.HTTPError as e:
            mb.showerror(f"HTTP-ошибка: {e.r.status_code}")
            return None
        except requests.RequestException as e:
            # Сюда попадут любые другие ошибки requests
            mb.showerror(f"Произошла ошибка при запросе: {e}")
            return None


    def on_timer(period):
        # Создает таймер и помещает его в поток. С периодичностью, заданной в секундах, параметр period,
        # функция посылает событие event_time
        global dem_th


        def interruptible_sleep(secs):
            # Функция сна, стостоящая из коротких периодов для оперативного реагирования на сигнал останова
            start_time = time.time()    # Определяем текущее время
            # Если период меньше заданного и не установлен флаг останова
            while (time.time() - start_time < secs) and (not d_stop_event.is_set()):
                time.sleep(0.1)         # Засыпаем на короткий промежуток времени


        def worker():
            # Таймер для работы в фоновом потоке
            nonlocal period
            while not d_stop_event.is_set():  # Выполняет цикл, пока не установлен флаг останова
```

```
            # Получаем текущее время и преобразовываем его в int
            dt = datetime.now()
            timestamp = int(dt.timestamp())  # Целое число секунд
            # Отправляем событие в главный поток
            root.event_generate("<<event_time>>", when='tail', state=timestamp)
            # Задержка перед следующим обновлением
            interruptible_sleep(period)
        # Запускаем поток
        dem_th = threading.Thread(target=worker, daemon=True)
        dem_th.start()
    # on_timer

    def on_event(event):
        # Обработчик события таймера. По сигналу таймера запускает заданную функцию
        update_favor()  # Заданная функция

    def on_menu_timer():
        # Функция управляет запуском и остановом таймера для обновления Избранного
        global dem_th, d_stop_event
        if dem_th == None:  # Если поток обновления избранного не запущен, то вызываем диалог для ввода периода таймера
            ptime = simpledialog.askinteger("Таймер", "Задайте период обновления\nизбранного в минутах:", initialvalue=0, minvalue=0, maxvalue=240)
            if ptime is not None:
                if 0 < ptime:
                    on_timer(ptime * 60)  # Запуксаем таймер с периодом согнала ptime * 60 секунд
                    mb.showinfo("Таймер", f"Таймер запущен с периодом срабатывания {ptime} минут")
                    return
                else:
                    mb.showinfo("Таймер", f"Таймер не установлен. Нельзя запустить\nтаймер с нулевым периодом срабатывания")
```

```
                return
            else:
                return
        else:
            answer = mb.askyesno("Таймер", "Вы хотите прервать работу таймера?")
            if answer:                  # Если ответ "ДА", то:
                d_stop_event.set()      # устанавливаем флаг останова потока
                dem_th.join()           # дожидаемся останова потока обновления избранного
                dem_th = None           # обновляем переменную потока
                d_stop_event.clear()    # очищаем флаг останова
                mb.showinfo("Таймер", "Таймер остановлен")
                return
            else:
                return
    # on_menu_timer


    """ Основной текст программы """


    MAIN_ICON = 'pcryptoboard.ico'                # Имя файла с иконкой программы
    TOTAL_COINS_FNAME ='data/total_coins.json'            # Имя файла с полным списком валют
    FAVOURITES_COINS_FNAME = 'data/indices_coins.json'   # Имя файла со справочником избранных валют
    FAVOURITES_ID_FNAME = 'data/favor_id.cfg'           # Имя файла с идентификаторами избранных валют
    TIME_STAMP_FNAME = 'data/time_update.cfg'           # Имя файла с датой и временем последнего обновления
    SELECTED_INDS = "data/selected_indices.cfg"         # Имя файла с номерами выбранных строк для Listbox
    REQUEST_TOTAL_COINS ="https://api.coinlore.net/api/tickers/"     # Адрес запроса для всех монет
    FAVOURITES_COINS = "https://api.coinlore.net/api/ticker/"        # Адрес запроса по выбранным монетам
```

```
    FAVOURITES_VIEW = "data/sel_ind_flag.cfg"          # Имя файла для флага выделения избранного


    menu_flag = True                                   # флаг видимости рамки, строки заголовка и меню окна
    fav_id = ''                                        # Строка с индексами избранных валют
    time_stamp = ''                                    # Текущее время
    locker = threading.Lock()                          # Объект блокировки ресурсов
    thrd = None                                        # Переменная для потока обновления Справочника валют
    stop_event = threading.Event()                     # Флаг останова потока обновления Справочника валют
    dem_th = None                                      # Переменная для потока обновления Избранного
    d_stop_event = threading.Event()                   # Флаг останова потока обновления Избранного


    # Главное окно программы
    root = Tk()
    root.title("Курсы криптовалют")
    # root.geometry(f"{root_scr_size[0]}x{root_scr_size[1]}")
    root.minsize(300, 50)  # Задал минимальный размер окна
    root.iconbitmap(MAIN_ICON)


    frame_root = Frame(root, bg='gray')
    frame_root.pack(fill="x", padx=10, pady=5)


    frame_footer = Frame(root)
    frame_footer.pack(fill="x", padx=10)


    # Основное меню
    menu_bar = Menu(root)
    root.config(menu=menu_bar)
```

```
    settings_menu = Menu(menu_bar, tearoff=0)
    menu_bar.add_cascade(label='Панель', menu=settings_menu)
    settings_menu.add_command(label="Выбрать монеты", command=on_favourites)
    settings_menu.add_separator()
    settings_menu.add_command(label="Выход", command=quit)

    update_menu = Menu(menu_bar, tearoff=0)
    menu_bar.add_cascade(label="Обновление", menu=update_menu)
    update_menu.add_command(label='Таймер избранного', command=on_menu_timer)
    update_menu.add_command(label='Избранное', command=update_favor)
    update_menu.add_separator()
    update_menu.add_command(label='Справочник монет', command=update_guide_dialog)

    # Создаем виджеты в главном окне
    on_dashboard()

    # Двойной клик по окну скрывает рамку, строку заголовка и меню окна
    root.bind("<Double-Button-1>", lambda event: hide_menu(event) if menu_flag else show_menu(event))

    # Обработка события таймера
    root.bind("<<event_time>>", on_event)

    # Связываем событие закрытия окна с вызовом функции quit
    root.protocol("WM_DELETE_WINDOW", quit)

    root.mainloop()
```

7. ЗАКЛЮЧЕНИЕ

Разработанное программное обеспечение «Курсы криптовалют» позволит упростить пользователям поиск информации о популярных криптовалютах и имеет коммерческий потенциал.

В перспективе программа может быть доработана для более удобного отбора пользователем криптовалют в избранное, в частности:

- добавить строку поиска по названию,
- заменить существующий вывод общего списка криптовалют в один столбец, выводом в несколько столбцов (название, символ, капитализация и др.) с возможностью оперативной сортировки по каждому столбцу.

В перспективе может быть доработано главное окно программы:

- добавить столбцы с дополнительной информацией (изменения, капитализация, объем и предложение)
- добавить графические символы избранных криптовалют.

Также может быть доработано автоматическое обновление избранного:

- период обновления сохранять в файлах настроек; при запуске программы из файла с настройками читать этот период и сразу запускать обновление по таймеру с указанным периодом.

Для реализации коммерческого потенциала необходимо, также, создать систему контекстной помощи пользователю и добавить пункт меню «О программе», при выборе которого пользователь сможет получить необходимую информацию о работе с программой.

Для оперативного изменения источника данных вместо сайта https://www.coinlore.com (в случае прекращения предоставления этим сервером данных для российских пользователей), адрес сервера для запросов, а также параметры запросов сохранять в файлах настроек в текстовом формате (для возможности ручного редактирования адресов и параметров запросов) вместо использования констант в коде программы.